

ERROR AND EXCEPTION IN JAVA

Custom Exception, throw and throws

Introduction

Java programs may face unexpected problems during execution. These problems are mainly represented as Errors and Exceptions. Errors are usually serious system or JVM problems, while Exceptions can often be handled by the application. Understanding their difference helps programmers handle problems effectively.

Error

An Error is a serious problem that normally occurs because of the JVM or the runtime environment. Errors are generally not caused by normal application logic and are usually not intended to be handled by ordinary application code.

For example, if a program repeatedly calls a method without stopping, the call stack can become full. This can result in `StackOverflowError`.

Examples of Errors

- `OutOfMemoryError` – occurs when the JVM cannot allocate enough memory.
- `StackOverflowError` – can occur because of excessive or infinite method recursion.
- `NoClassDefFoundError` – can occur when a required class cannot be found at runtime.

Important Point

Errors are usually more serious than ordinary exceptions. The programmer should focus on preventing application errors where possible rather than assuming that every Error can be safely recovered from.

ERROR AND EXCEPTION IN JAVA

Exception

An Exception is an abnormal condition that occurs during program execution and can often be handled by the programmer. Java provides mechanisms such as try, catch, finally, throw, and throws to manage exceptions.

For example, dividing an integer by zero causes `ArithmeticException`. Instead of allowing the program to terminate unexpectedly, the exception can be caught and a suitable message can be displayed.

Common Exceptions

- `ArithmeticException` – occurs during an invalid arithmetic operation such as integer division by zero.
- `NullPointerException` – occurs when an operation is performed on a null reference.
- `ArrayIndexOutOfBoundsException` – occurs when an invalid array index is accessed.
- `IOException` – represents input/output related problems.

Difference Between Error and Exception

ERROR	EXCEPTION
Usually related to JVM or system-level problems.	Usually related to problems that occur during program execution.
Generally serious.	Can often be handled by the application.
Usually not expected to be recovered from.	Often possible to recover from or report properly.
Programmers normally do not handle Errors.	Programmers commonly handle Exceptions.
Examples: <code>OutOfMemoryError</code> , <code>StackOverflowError</code> .	Examples: <code>IOException</code> , <code>ArithmeticException</code> .
Can indicate a serious runtime failure.	Can represent an expected or manageable abnormal condition.

Exception Handling

Exception handling allows a Java program to respond to an abnormal situation without immediately stopping the entire application. The try block contains code that may cause an exception, while the catch block contains code that handles it.

Program

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Output: Cannot divide by zero

CUSTOM EXCEPTION

What is a Custom Exception?

A custom exception is an exception class created by the programmer for a specific application requirement. It is useful when the standard Java exceptions do not clearly describe the condition that the program wants to report.

In this assignment, MyException is created by extending the Exception class. The constructor receives a message and passes it to the parent Exception class using super(message).

MyException.java

```
package customexceptionio;

public class MyException extends Exception {

    private static final long serialVersionUID = 1L;

    public MyException(String message) {
        super(message);
    }
}
```

Explanation

The statement extends Exception makes MyException a checked exception. The constructor accepts a message such as "Student is Fail". The super(message) statement sends that message to the Exception class so it can later be obtained using getMessage().

throw Keyword

The throw keyword is used to explicitly create and throw an exception at a particular point in the program.

```
throw new MyException("Student is Fail");
```

In this program, if the student's mark is less than 40, the custom exception is thrown with the message "Student is Fail".

throws Keyword

The throws keyword is written in a method declaration to indicate that the method may throw a particular exception. It transfers the responsibility of handling the exception to the calling code.

```
static void checkMarks(int marks) throws MyException
```

Difference Between throw and throws

throw	throws
Used to explicitly throw an exception.	Used in a method declaration.
Used inside the method body.	Written after the method parameters.
Throws one exception object at a time.	Can declare one or more exception types.

Main.java

```
package customexceptionio;

public class Main {

    static void checkMarks(int marks) throws MyException {

        if (marks < 40) {
            throw new MyException("Student is Fail");
        } else {
            System.out.println("Student is Pass");
        }
    }

    public static void main(String[] args) {

        try {
            checkMarks(90);
        } catch (MyException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Program Explanation

Step 1: The checkMarks() method receives the student's marks.

Step 2: The method declaration contains throws MyException because the method may throw the custom exception.

Step 3: The if condition checks whether the marks are below 40.

Step 4: If marks are below 40, throw new MyException("Student is Fail") creates and throws the custom exception.

Step 5: If marks are 40 or above, the program prints "Student is Pass".

Step 6: In main(), the method call is placed inside a try block.

Step 7: The catch block catches MyException and displays the exception message.

Output

```
Student is Pass
```

The program uses checkMarks(90), so the condition marks < 40 is false. Therefore, the program prints "Student is Pass".

Testing the Custom Exception

If the value is changed from checkMarks(90) to checkMarks(30), the condition becomes true and the custom exception is thrown.

```
Student is Fail
```

This demonstrates that the custom exception works according to the condition defined by the programmer.

File I/O Example

File I/O means File Input and Output. Java provides classes such as `FileWriter` and `FileReader` to write data to a file and read data from a file. `IOException` is handled using try-catch.

FileIoExample.java

```
package customexceptionio;

import java.io.FileWriter;
import java.io.FileReader;
import java.io.IOException;

public class FileIoExample {

    public static void main(String[] args) {

        try {
            FileWriter writer = new FileWriter("data.txt");
            writer.write("Sharon Emile");
            writer.close();

            FileReader reader = new FileReader("data.txt");

            int ch;
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }

            reader.close();

        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

File I/O Output

```
Sharon Emile
```

The `FileWriter` creates or writes to `data.txt`. The `FileReader` then reads the characters one by one. The while loop continues until the end of the file is reached.

Conclusion

Error and Exception are important in Java. Errors are usually serious system problems, while Exceptions can often be handled. This program demonstrates a custom exception using “throw” and “throws”, along with basic File I/O using “FileWriter” and “FileReader”.